

National Institute of Technology, Rourkela
End Semester Examination, Autumn 2023
Compiler Design (CS3007)

5th semester B.Tech in Computer Science & Engineering

Duration: 3 Hours
Number of pages: 2

Full Marks: 50

- Attempt all questions. Q1 – Q3 are in page 1, Q4 and Q5 are in page 2.
- If you answer a question, all sub-questions should form part of it.

- Q 1. (a) Construct a DFA directly for the regular expression $(a|b)^*abb$ by computing *nullable()*, *firstpos()*, *lastpos()*, and *followpos()* for the corresponding syntax tree.
- (b) Consider following grammar which generates email addresses:
 $\{Addr \rightarrow Name@Name.id$
 $Name \rightarrow id|id.Name\}$
For example, this could generate the addresses $id@id.id$ $id.id@id.id.id.id$ $id.id.id@id.id$
This grammar, as written, is not LL(1). (i) Rewrite the grammar to eliminate all LL(1) conflicts. (ii) Construct the LL(1) parse table for your updated grammar. (6 + 4)
- Q 2. (a) Write the *quadruples*, *triples*, and *indirect triples* for the expression $-(a + b) + (c + d) - (a + b + c)$.
- (b) Consider the following Syntax-Directed Translation for desk calculator. Draw the annotated parse tree for $23*5+4\$$

Production	Semantic Actions
(1) $S \rightarrow E\$$	{print E.VAL}
(2) $E \rightarrow E_1 + E_2$	{E.VAL = $E_1.VAL + E_2.VAL$ }
(3) $E \rightarrow E_1 * E_2$	{E.VAL = $E_1.VAL * E_2.VAL$ }
(4) $E \rightarrow (E_1)$	{E.VAL = $E_1.VAL$ }
(5) $E \rightarrow I$	{E.VAL = I.VAL}
(6) $I \rightarrow I_1 digit$	{I.VAL = $10 * I_1.VAL + LEXVAL$ }
(6) $I \rightarrow digit$	{I.VAL = LEXVAL}

and evaluate it.

(6 + 4)

- Q 3. Consider the following matrix multiplication routine:

```

1: for i = 1 to n do
2:   for j = 1 to n do
3:     C[i, j] = 0;
4:   end for
5: end for
6: for i = 1 to n do
7:   for j = 1 to n do
8:     for k = 1 to n do
9:       C[i, j] = C[i, j] + A[i, k] * B[k, j];
10:    end for
11:  end for
12: end for

```

- (a) Assume A, B, and C are allocated static storage and there are two bytes per word in a byte-addresses memory, produce three-address code for the above routing.
- (b) Partition the generated three-address codes into *basic blocks* and identify the *leaders*
- (c) Find the loops in the *flow graph* and move the *loop invariant computations* out of the loops. (5 + 2 + 3)

Q 4. (a) Apply loop unrolling on the following code fragment to optimize it.

```

1: I = 1
2: while I ≤ 100 do
3:   A[I] = 0;
4:   I = I + 1;
5: end while

```

(b) Apply loop jamming on the following code fragment to optimize it.

```

1: for I = 1 to 10 do
2:   for J = 1 to 10 do
3:     A[I, J] = 0;
4:   end for
5: end for
6: for I = 1 to 10 do
7:   A[I, I] = 1;
8: end for

```

(c) A real array $A[i, j, k]$ has index i ranging from 1 to 40, j ranging from 0 to 4, and k ranging from 5 to 10. Reals take 8 bytes each. If A is stored row-major, starting at 0 byte, find the location of $A[4, 3, 9]$.

(d) Write the Action parts for the following Syntax-Directed Translation to print from infix-to-postfix.

Production	Action
$E \rightarrow E_1 op E_2$	{?}
$E \rightarrow (E_1)$	{?}
$E \rightarrow id$	{?}

(3 + 3 + 2 + 2)

Q 5. (a) Draw the Expression Tree for $(A * (B/C)) + (D - (E + F))$.

(b) Label the nodes of the tree to generate the optimal target code.

(c) Generate the optimal machine codes for the expression where the available registers set is $\{R_0, R_1\}$. The available instructions are as follows:

ST M_i, R_j	// $M_i = R_j$
LD R_i, R_j	// $R_i = R_j$
LD R_i, M_j	// $R_i = M_j$
ADD R_i, R_i, R_j	// $R_i = R_i + R_j$
SUB R_i, R_i, R_j	// $R_i = R_i - R_j$
MUL R_i, R_i, R_j	// $R_i = R_i * R_j$
DIV R_i, R_i, R_j	// $R_i = R_i / R_j$
LD R_i, R_j	// $R_i = R_j$
ADD R_i, R_i, M_j	// $R_i = R_i + M_j$
SUB R_i, R_i, M_j	// $R_i = R_i - M_j$
MUL R_i, R_i, M_j	// $R_i = R_i * M_j$
DIV R_i, R_i, M_j	// $R_i = R_i / M_j$

(2 + 2 + 6)

*****All the best*****